M256 Unit 9
UNDERGRADUATE COMPUTING

Software development with Java

Detailed design

Unit 9

This publication forms part of an Open University course M256 *Software development with Java.* Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom:
tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5482 7

2.1

CONTENTS

1 Introduction 5

1.1 Completing the Hospital System implementation model 5

1.2 The aims of the unit 7

1.3 Studying this unit 7

2 Specifying types 8

2.1 Link references and method answers 8

2.2 Attributes 10

3 Specifying the utility class `Name` 14

3.1 Representing names 14

3.2 `equals()` for `Name` 17

3.3 `compareTo()` for `Name` 18

3.4 `hashCode()` for `Name` 20

3.5 `toString()` for `Name` 22

3.6 Applying good practice to core system classes 23

4 Refactoring 26

4.1 The `Person` class 26

4.2 Inheritance 28

4.3 Composition 29

4.4 Composition and inheritance compared 32

4.5 `hospitalcore` and `m256people` 37

5 Summary 38

Glossary 39

Index 40

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Compositor

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

Previous units have introduced you to some key phases of software development. Working with the main example of the Hospital System, you considered requirements specification in *Unit 2*, developed a conceptual model and an initial structural model of the core system in *Units 3* and *4*, and then designed dynamic models in *Units 6*, *7* and *8*. For each class you reached decisions such as:

- which attributes should be specified;
- to which other objects the objects of the class should be linked;
- which methods should be specified.

These decisions were documented in the structural part of the evolving implementation model for the core system. Recall that this structural part or *structural model* consists of a class diagram and associated text (class descriptions and invariants). The implementation model also has a *dynamic part* (dynamic models of the system in action based on walk-throughs and sequence diagrams).

1.1 Completing the Hospital System implementation model

In this unit we consider some decisions that still need to be taken before the Hospital core system can actually be implemented. This will complete the Hospital System's implementation model, that is, the final design upon which the core system code and its testing (to be dealt with in *Unit 10*) will be based. As implementation approaches, so the implementation model naturally gets increasingly Java related (Java being our target language). However, the design ideas expressed by the implementation model are still sufficiently general that they could be reused on similar projects, including those with different target languages, while being sufficiently informative that the implementation model could be handed over from designer to programmer, providing a 'blueprint' for the latter to work from.

You will be considering the theory behind some of the possible choices in this detailed phase of design and also how these choices would affect the Hospital System implementation model. Figure 1 shows how this detailed design phase fits into the larger picture of software development.

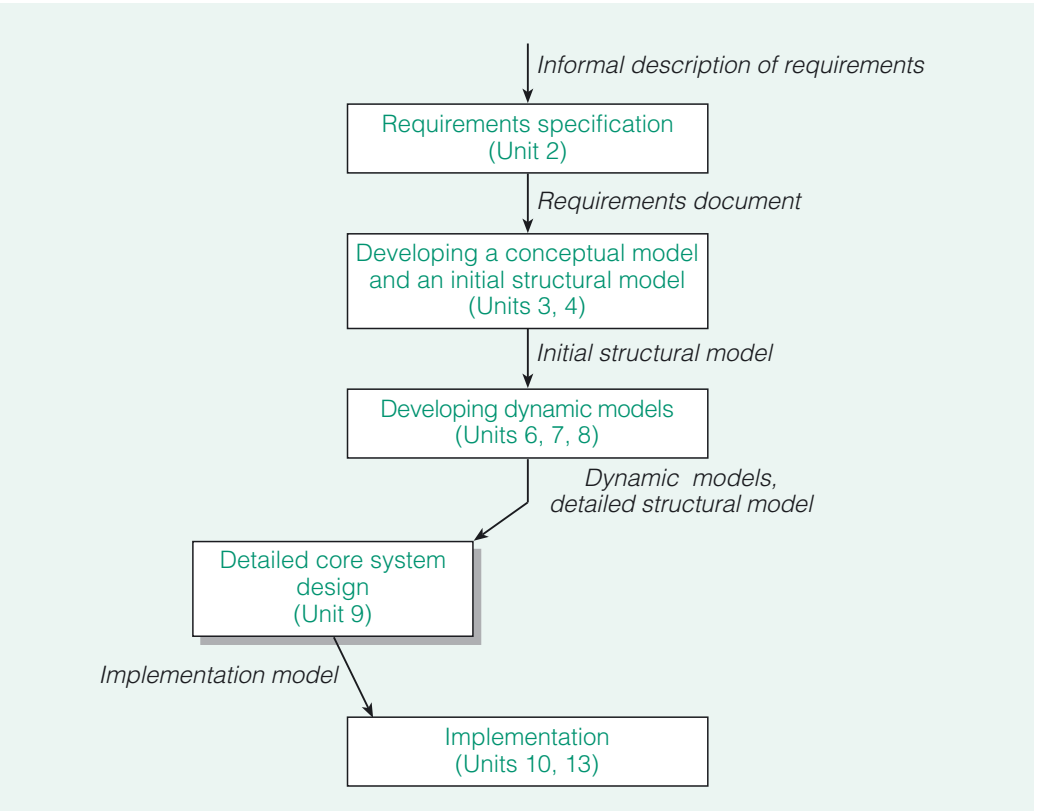


Figure 1 The place of detailed design in the software development process

The decisions to be made in this unit are informed by two guiding principles that reflect the key aspects of software quality described in *Unit 5*:

1 to improve the maintainability of the system;

2 to increase reuse of software components.

SAQ 1

In *Unit 5* two aspects to component-based software development were identified: *design with reuse* and *design for reuse*. Briefly explain the meaning of each of these terms.

ANSWER.....

Design with reuse is concerned with using existing components in creating a new piece of software.

Design for reuse is concerned with producing components as part of developing new software, or specifically as marketable commodities.

The simplicity of the Hospital System will mean that there are no components from previous developments that can be reused (except for the `m256date` package which we have provided for your use throughout the course). Of course, the standard packages supplied by the Java API will be reused, for example, the Collections Framework from `java.util`.

1.2 The aims of the unit

The main focus of this unit is on reviewing the structural part of the implementation model as it has been developed so far. Consideration will be given to improving the structure to make the system more maintainable, and to both increasing reuse within the system *and* enhancing the potential for parts of the system to be reused elsewhere in the future. We are aiming to give an insight into typical issues that would arise at this stage of development and to investigate some key principles, rather than to provide an exhaustive guide.

In detail, in this unit the following issues will be discussed:

- how the existing Java API can be used appropriately to specify types for link references, method answers and attributes (Section 2);
- the specification of a utility class (Sections 3 and 4);
- the idea of refactoring a design (Section 4).

1.3 Studying this unit

The unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise and read the solutions provided before moving on.

This unit is shorter than others in the course and should take less time. There is no practical work on the computer. Sections 3 and 4 are the most substantial sections of this unit, requiring approximately equal study time. Section 2 might require approximately half the study time of Section 3.

You will need to refer to the *Handbook* during your study of this unit. You may also like to briefly review the Appendix to *Unit 7* before reading on, as it contains the evolving Hospital System structural model which is the starting point for this unit. Exercise 12 requires you to refer to the Appendix to *Unit 7*; other than this, those aspects of the structural model which you actually need to work with are included in this unit.

2 Specifying types

In this section some issues relating to the specification of types are considered. To be precise, you will study what kinds of object or data value should be used to implement:

- link references;
- method answers;
- attributes.

These sorts of issue have of course been raised earlier in the design process. Here the designs are reviewed, considering the issue of types more generally.

2.1 Link references and method answers

As you learnt in earlier units, in the working system a link will generally be implemented by an object holding a reference to some other object. The reference is stored in an instance variable. Our previous design work has already generated certain information about these instance variables. For example, here is an excerpt from the `Ward` class description. As well as attributes, `Ward` has a responsibility for holding link references, listed under **Links**. It also has a related method, listed under **Protocol**.

Class `Ward` — A hospital ward

Attributes

`private String name`	The unique name of the ward
`private Sex type`	Whether the ward is for male or female (M or F) patients
`private int capacity`	The maximum number of patients that can be on the ward at any one time
[`int/numberOfFreeBeds`	The number of free beds on the ward]

Links

`private Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

Protocol

`Collection<Patient> getPatients()`

Post-condition: returns a collection of all the linked `Patient` objects.

A link may be to one object or many. In the first case – one object – all that is needed is an instance variable to store a reference to that object. In the second case a reference to a collection will be needed. All programming languages support some form of collection. Java in particular has the classes that implement the interface `Collection`, and these are what we use in M256 to implement links to multiple objects. Each time a link reference is to several objects, the type of the instance variable is specified as a `Collection` interface, e.g. `Collection<Patient>`. Similarly, whenever a method returns a collection of several objects, the return type of the method answer is a `Collection` interface. We expect you will have encountered various Java interfaces in your previous programming work.

SAQ 2

(a) What is a Java interface?

(b) What is required of a class which implements a given interface?

(c) Give an example of an interface other than `Collection`, and of a class which implements it.

ANSWER.....

(a) A Java interface is a specification of a group of methods, which for each method simply declares the method header and no method body.

(b) A class which implements a given interface has to provide or inherit a definition for each method specified in the interface. That is, it has to provide a method body. (Strictly, this is true only for a concrete, i.e. non-abstract, class. An abstract class implementing an interface need not implement a method specified in the interface. In this case, in the abstract class the method becomes an abstract method.)

(c) There are many examples you might have chosen. `Collection` itself has many subinterfaces, that is, interfaces that extend the group of methods specified by `Collection`. The `Set` interface is one such example; `HashSet` is a class which implements this interface.

Alternatively, you might have thought of the `Map` interface, which defines collections of key–value pairs and is implemented by `HashMap`, among other classes.

In using the Java collections it is considered good practice to specify in the design, and then declare in code, the type of a variable as an interface, where an appropriate interface exists. For example, rather than declaring the instance variable `patients` in the class `Ward` as being a specific kind of collection (`HashSet`, for example), it is better to declare it as being of type `Collection<Patient>`. Such a type is referred to as the **interface type** of the variable. The actual class of object that the variable can reference is then restricted by the interface type.

SAQ 3

A variable of interface type `A` references an object of the class `B`. How is `B` related to `A`?

ANSWER.....

The class `B` must implement the interface `A`.

From the answer to SAQ 3 it is apparent that, in the working system, each instance variable that represents a link reference to a collection will actually reference an object of a class that implements the interface specified in the class description. For example, you saw above that the design of `Ward` included a link reference, `patients`, declared as being of the interface type `Collection<Patient>`. In *Unit 10* you will see that at implementation we arrange for `Ward` to have an instance variable, `patients`, which references an instance of `HashSet<Patient>`, a class that implements `Collection<Patient>`.

Similarly, each method that returns a collection will return an object of a class that implements the interface specified in the class description. It is naturally tempting to ask the question: 'Which class? Shouldn't the class description specify this?'

The answer is an emphatic no! The decision should be left to the programmer, who may know of good reasons – faster or more efficient execution say – for using one class rather than another. The designer specifies, by their choice of interface type, the behaviour that is required of any class of object that the instance variable references;

after that, the programmer employs their expertise in determining an appropriate class of object to use in the implementation.

Moreover, if the designer specifies only the interface, it will later be possible, if required, to replace the implementation with a different one, while maintaining the same interface. Since existing clients will have assumed only what is specified in the interface, they will not be affected by the change. Consider the following method specified for the Hospital System coordinating class `HospCoord`.

```
public Collection<Patient> getPatients(Ward aWard)
```

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

A client invoking this method knows only of the interface type, `Collection<Patient>`, specified for the method answer. Consequently, all that the client can rely on is being able to invoke methods on the value returned that are specified in the `Collection` interface (i.e. methods applicable to all collections, such as `isEmpty()` and `size()`). So, in the core system implementation, the actual class may be changed to another class that implements the same interface, without affecting the client, from whose perspective the core system remains unchanged.

2.2 Attributes

As with link references, attributes will generally be implemented by instance variables, each of which will have the same name as the attribute concerned. For each such instance variable the type of object or data value that it should reference must be specified in the class description. We made provisional decisions in *Units 6* and *7* in order that certain methods could be specified, but until now we have not considered the issue of types per se. For example, in *Unit 6*, we specified the coordinating method for the *Admit Patient* use case with a method header as follows:

```
public Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

At the time we said that we were anticipating the specification of the following classes:

- `Name`, an instance of which represents the name of a patient;
- `Sex`, an instance of which represents the sex of a patient.

At that earlier design stage, we specified existing Java types, such as `int` and `String`, for attributes, where they seemed obvious. Where it was not obvious what type was appropriate, we invented one, such as `Name` or `Sex`, anticipating the specification of that type when more was known about how it is used.

Here is part of the class description for `Patient`, as it stood at the end of *Unit 7*.

Class `Patient`	A patient in the hospital
Attributes	
`private Name name`	The name of the patient
`private Sex /sex`	The sex (M or F) of the patient
`private M256Date dateOfBirth`	The date of birth of the patient
`[int /age`	The age of the patient in years]

SAQ 4

What do each of / and [...] in the specification of the attribute `age` signify?

ANSWER.....

The / signifies that `age` is a derived attribute. That is, its value may be derived from other aspects of the model. The derivation is based on invariant 2 in the Hospital System initial structural model:

> For each `Patient` object, the value of its `age` attribute is equal to the difference (in years) between the current date and the value of its `dateOfBirth` attribute.

The [...] signifies the design decision that the attribute `age` would *not* be implemented as an instance variable. Instead, the system will derive the value of the attribute whenever it is needed.

At this stage, the nature of the information that each attribute records is reviewed, confirming for some attributes the provisional choice of Java types and, for others, specifying **user-defined types** such as `Name` and `Sex`. The information may be simple – a single number, character or string, for example. Or it may be composite – a combination of several numbers, characters and/or strings, for example. Any restrictions on the values must also be considered. For example, the sex of a patient may be only male or female.

For the Hospital System the attributes that are to be implemented as instance variables are the following.

`name`, `type`, `capacity` (in `Ward`)
`code` (in `Team`)
`name`, `sex`, `dateOfBirth` (in `Patient`)
`name` (in `Doctor`)
`grade` (in `JuniorDoctor`)

The Hospital System requirements document gives some information about the values of the real-world properties that are modelled by attributes. Example team codes are quoted, for instance, such as ‘Orthopaedics A’ or ‘Paediatrics’. For the purposes of this unit we will invent some plausible kinds of value for other attributes. In reality, the requirements document would generally give fuller information about values; otherwise, the client would be consulted on these matters.

Suppose that the following table gives some typical values for these attributes. The final column shows our provisional choices for instance variable types, as set out in the Appendix to *Unit 7*.

Information	Typical values	Type?
`name` in `Ward`	Brookfield, Grange Spinney	`String`
`type` in `Ward`	male, female (only)	`Sex`
`capacity` in `Ward`	10, 12, 15	`int`
`code` in `Team`	Orthopaedics A, Paediatrics	`String`
`name` in `Patient`	Ms Lynda Snell, Mr Mike Tucker	`Name`
`sex` in `Patient`	male, female (only)	`Sex`
`dateOfBirth` in `Patient`	21/11/43	`M256Date`
`name` in `Doctor`	Ms Susan Carter, Mr Bert Fry	`Name`
`grade` in `JuniorDoctor`	1, 2 or 3 (only)	`Grade`

Looking at the typical values, you can confirm the earlier choices of existing Java types. For example, `name` (in `Ward`) and `code` (in `Team`) can indeed be implemented by instance variables of type `String`, since the typical values are short, simple pieces of text.

Exercise 1

Justify the choice of `int` as the type of the instance variable `capacity` in `Ward`.

Discussion

The type `int` is suitable because the typical `capacity` values are whole numbers. You might also have noted that `int` is appropriate because arithmetic operations will need to be performed on `capacity` values (to derive the values of `numberOfFreeBeds`).

Enumerated types

Now consider how to represent the sex of a patient. Your initial thoughts might be to use `char` or perhaps `String` for this, but the `sex` attribute should be restricted to taking just two values, representing male and female. It would be possible to achieve this using `char`, for example, by incorporating checks in the code to ensure that the `sex` instance variable takes only the `char` values `'M'` and `'F'`.

However, Java provides an easier way: specifying the type of `sex` as `Sex`, where `Sex` is not a class, but an **enumerated type** (an **enum**). An enumerated type can take on only one of a finite set of values, which are specified by the programmer when the type is defined. Attempting to set a variable of an enumerated type to an unspecified value is illegal: such code will not compile. You will see exactly how an enumerated type is defined in code in *Unit 10*. For now all that is required is to specify `Sex` as an enum with values representing male and female, and to specify `Sex` as the type of `sex` in the `Patient` class description.

Since the `type` attribute of `Ward` has values that are restricted in exactly the same way as for the `sex` attribute of `Patient`, it is also appropriate to specify `type` as an instance variable of type `Sex`.

Here is the specification of `Sex`:

Enum `Sex` The sex of a person

Values `M` and `F`

Exercise 2

Specify a suitable type, `Grade`, for the instance variable `grade` which implements the `grade` attribute of `JuniorDoctor`.

Discussion

The instance variable `grade` should be restricted to taking values representing the grades 1, 2 and 3 only. A suitable type could be an enum `Grade`, specified as follows.

Enum `Grade` The grade of a junior doctor

Values `1`, `2` and `3`

In fact, integers are not permitted to represent values of an enumerated type (the values of an enumerated type have to be represented by valid Java identifiers), so we will define `Grade` as follows:

Enum `Grade` The grade of a junior doctor

Values `ONE`, `TWO` and `THREE`

It remains for us to consider the ways in which the instance variable `name` in the classes `Patient` and `Doctor` could be implemented. This will be done in the next section. The names of doctors and patients will be treated differently from the names of wards, since they are quite different entities. The names of people, as you will see, have a distinct, complex structure, whereas the names of the wards (at least in the Hospital System) do not: they are appropriately represented simply by instances of `String`.

In this section you learnt how to use both existing Java types and enumerated types for instance variables that implement attributes. You also saw that, where collections are involved, interface types should be used for instance variables that implement link references and for method answers.

3 Specifying the utility class `Name`

At the end of Section 2 you were introduced to the concept of an enumerated type, which is a kind of user-defined type. This section will continue to look at user-defined types by:

- considering the representation of names;
- specifying the utility class `Name`;
- introducing some general principles of good practice when specifying a utility class.

3.1 Representing names

For the immediate purposes of our system it would be sufficient simply to represent people's names by instances of the Java type `String`. But to introduce you to some of the issues involved when developers attempt to anticipate the needs of future projects, we will consider some other possibilities.

When filling in forms, people are rarely asked simply for a name. Usually separate pieces of information are required: a surname, a first name, a title (Mr, Ms, Mrs, ...) etc. Of the many ways in which this could be represented in code, here are three.

1 Use an instance of `String` to record all these separate pieces of information, separating them with a special character such as `&`. For example `"Ms&Lynda&Snell"`.

2 Use an array in which each element refers to an instance of `String` representing one of the parts of the name. Figure 2 shows an example of an array object, `array1`, with three such elements.

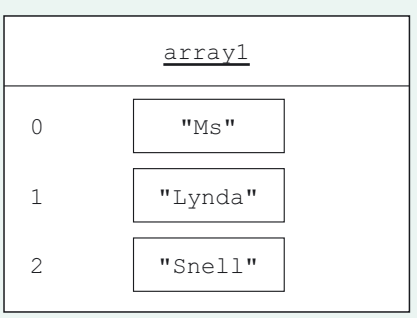


Figure 2 Using an array to represent a name

3 Define a new class, `Name`, in which the various parts of a name are represented by instance variables of `Name`, which themselves will reference instances of `String` (see Figure 3).

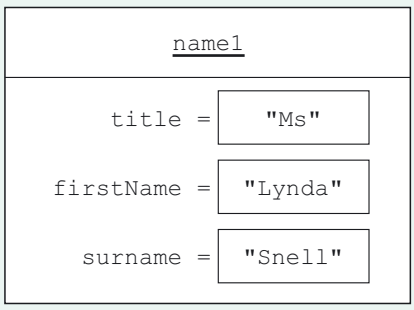


Figure 3 Using a purpose-built class to represent names

The third is the best object-oriented approach of the three, as was anticipated (but not detailed) earlier in the design process. It packages the separate pieces of information as a single purpose-built object that could be reused in other systems. Such a class, which is not specific to the particular system under development, but which has potentially wider applicability, is termed a **utility class**. We will take this approach for the attributes `name` in `Patient` and `Doctor`, and will specify the utility class `Name`.

The need to work with names is very common in computer systems, and it is possible that `Name` could be reused in many different settings, with different requirements. It is impossible to predict exactly what might be required of `Name` in future, so a sensible approach is to equip the class with a small, well-documented, cohesive set of services. Future developers can then:

- understand the class (it is not overly complex);
- have flexibility to reuse it in various ways.

In reality, for a class such as `Name` to be a genuine candidate for reuse, it would require more complex functionality than can be discussed here.

For example, it is likely that the individual parts of a name will need to be retrieved; consequently it is appropriate to specify getter methods for the instance variables `title`, `firstName` and `surname`, resulting in the following specification.

Class `Name` — A person's name

Attributes

`private String title`	The title of the person
`private String firstName`	The first name of the person
`private String surname`	The surname of the person

Protocol

`public String getTitle()`

Post-condition: returns `title`.

`public String getFirstName()`

Post-condition: returns `firstName`.

`public String getSurname()`

Post-condition: returns `surname`.

SAQ 5

Why are the attributes specified as `private`, whereas the methods in the protocol are specified as `public`?

ANSWER

Specifying attributes as `private` means that the instance variables which implement them will have private accessibility. This is consistent with the good practice of preventing clients from obtaining direct access to the instance variables which implement the attributes.

The methods in this protocol are getter methods, designed to enable clients to find out about the attribute values, so they have public accessibility.

It should be clear that what is being specified for the `Name` class goes beyond the immediate requirements of the system at hand. The Hospital System requirements make no mention of distinguishing titles and first names etc. (though in fact we will later opt to have the user interface make use of some of this functionality). You must ensure that when you equip a class with services likely to be useful to future projects, you do not render the class too complex for the current project. With `Name`, the situation is quite straightforward because `Name` objects are used in the Hospital System in relatively simple ways:

- as attribute values (for example, `name` in `Patient`);
- as return types (for example, the return type of the method `getName()` in `Patient`);
- as types of method argument (for example, the `HospCoord` method with header `admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)`).

Because `Name` is used in relatively simple ways there is no problem with it having more functionality than is immediately required for the Hospital System: the extra functionality can simply be ignored. In Section 4, though, you will see how adding functionality to a class that is used in more complex ways can have an impact on the current system.

Exercise 3

Other than accessing the individual parts of a name, what services might future clients require of the `Name` class?

Discussion

There is no one correct answer to this question because anticipating the needs of future clients is not an exact science. Here are two additional services we thought of, which will be discussed further below:

- a way of determining whether two `Name` objects represent the same name;
- a means of comparing `Name` objects so they can be ordered alphabetically.

In answering Exercise 3 you might have considered equipping the `Name` class with a means to change the individual parts of a `Name` object, that is, specifying setter methods for the instance variables of `Name`. However, we take the view that it is appropriate to consider a name as an unchanging entity, for the following reason. Consider what is meant when it is said that a person's name has 'changed'. What is really meant is that the person is assigned a *new* name. The old name still exists (and is often still relevant information about that person). On this basis it is not appropriate for a `Name` object to allow, say, its `surname` value to be changed. If a person's surname 'changes', then this should be represented by the creation of a new `Name` object with that surname.

Consequently, Name will be specified to be an immutable class, i.e. one whose instances are immutable objects and cannot be modified. It is important to note that this does not mean that the name attribute of a Patient object cannot be changed. A different Name object can be assigned to a Patient object's name instance variable; what cannot be done is to modify the Name object which is referenced by name.

The concept of immutability was mentioned in *Unit 5*, Subsection 6.1.

Above, we have assumed that there will be a means of creating a new Name object; this will of course generally be required by any client of the class. Specifying a means of creating Name objects is straightforward. The following is added to the Name class description, under the heading **Constructor**.

```
public Name(String aTitle, String aFirstName, String aSurname)
```

Post-condition: initialises a new Name object with the given attribute values.

We will now consider the services noted in the discussion to Exercise 3.

3.2 equals() for Name

The method equals() is inherited by all classes from Object, where it is defined as follows:

```
public boolean equals (Object obj)
{
    return (this == obj);
}
```

From this code you can see that, unless the designer of a class specifies a different equals() method, the class will inherit an equals() method such that a.equals(b) tests whether a and b reference the same object. That is, it tests object identity. Very often, there is a more natural meaning of equality, different from identity, that should be applied to objects of that class. In such a situation an equals() method should be specified for the class accordingly. This is the case with Name.

Exercise 4

Two names are usually considered to be the same if their title, first name and surname are the same. On this basis, complete the following specification of the method equals() for the Name class.

```
public boolean equals(Object o)
```

Post-condition: returns true if o is a Name object with _____ _____ ; otherwise returns false.

Discussion.....

```
public boolean equals(Object o)
```

Post-condition: returns true if o is a Name object with title, firstName and surname equal to those of the receiver; otherwise returns false.

By specifying an equals() method as in the discussion to Exercise 4, distinct Name objects will be considered equal if they have the same state.

3.3 compareTo() for Name

Names are generally ordered alphabetically – think of names in a telephone directory. The name Ms Lynda Snell, for instance, would be considered to precede the name Ms Brenda Spall, since the surname Snell is alphabetically before the surname Spall. There are other possible orderings for names but alphabetical ordering is the predominant one.

SAQ 6

(a) Does the name Ms Carron Kemp precede the name Mr Brian Kemp in the usual ordering of names?

(b) Does the name Mr Hilary Hunter precede the name Ms Hilary Hunter in the usual ordering of names?

ANSWER.....

(a) No: the name Mr Brian Kemp precedes the name Ms Carron Kemp. When surnames are the same, names are usually ordered according to alphabetical order of first name, and the first name Brian precedes the first name Carron.

(b) Yes: the name Mr Hilary Hunter precedes the name Ms Hilary Hunter. When both surname and first name are the same, names are usually ordered according to alphabetical order of title.

Corresponding to this usual ordering, a method `compareTo(aName)` will be defined for `Name`. It will return a value signifying whether the receiver is alphabetically less than, equal to or greater than `aName`.

In fact, this decision is an application of general good practice when designing classes for reuse. Where there is a natural ordering on the objects of a class, a method `compareTo()` should be defined, to enable objects of the class to be compared. This method is the sole method specified in the interface `Comparable` (in `java.lang`), which should be implemented by classes whose objects can be compared. The reason for this is that Java collections which order and sort elements (think of `TreeSet` etc.) do so by assuming that those elements are of a class implementing `Comparable`, so that the elements have a method `compareTo()` by which they can be compared.

So to make it easy for developers to create collections of `Name` objects that can be sorted into their natural order, the `Name` class should implement `Comparable<Name>` and the method `compareTo()` should be specified for it. To decide *how* to specify `compareTo()` for `Name`, consider Figure 4, which shows an excerpt from the specification of the method `compareTo()` in the interface `Comparable`.

compareTo

`int compareTo(T o)`

Compares this object with the specified object for order. Returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.

Figure 4 An excerpt from the specification of `compareTo()`

SAQ 7

What is meant by saying that one Name object, name1, is less than another Name object, name2?

ANSWER.....

This means that name1 is alphabetically before name2, where the objects are compared using surname, firstName and title in that order.

Here is a suitable specification for compareTo() for Name.

```
public int compareTo(Name aName)
```

Post-condition: returns a negative integer if the receiver is alphabetically before aName, a positive integer if the receiver is alphabetically after aName and zero otherwise. Name objects are compared using surname, firstName and title in that order.

Keep in mind that we are still in the realm of design. We are not actually writing any code at this design stage: we are specifying what the programmer is to write, but not how.

What exactly is meant by describing this as a suitable specification of compareTo() for Name? Well, not only does this specification conform to the specification of the method compareTo() in Comparable, but it is also, as you will see, *consistent* with the specification of Name's equals() method, as is strongly recommended in the specification of the interface Comparable, the relevant part of which is reproduced in Figure 5.

> It is strongly recommended (though not required) that natural orderings be consistent with equals. This is so because sorted sets (and sorted maps) without explicit comparators behave "strangely" when they are used with elements (or keys) whose natural ordering is inconsistent with equals.

Figure 5 Consistency of a natural ordering with equals()

An ordering being 'consistent with equals' means that whenever a and b are such that a.compareTo(b) returns 0, then a.equals(b) must return true, and vice versa. This consistency is important when working with sorted collections which treat a and b as equal if a.compareTo(b) returns 0.

For the purposes of this unit you do not need to know any further details of how sorted collections use compareTo().

In most cases, specifying equals() to match the 'natural' equality of objects (if one exists) and compareTo() to match the 'natural' ordering of objects (if one exists) automatically leads to consistency.

SAQ 8

Explain why, with a and b referencing Name objects, if a.compareTo(b) returns 0 then it must be the case that a.equals(b) returns true, and vice versa.

ANSWER.....

The specification of compareTo() for Name guarantees that if a.compareTo(b) returns 0 then a is neither before nor after b alphabetically. Since alphabetic comparison compares surname, firstName and title, each of these instance variables must have the same value in the objects referenced by a and b. So, from the specification of equals() for Name, a.equals(b) returns true.

Conversely, if a.equals(b) returns true, then from the specification of equals() in Name, b has title, firstName and surname equal to those of a. It then follows from the specification of compareTo() for Name that a.compareTo(b) returns 0.

3.4 hashCode() for Name

Another general principle applicable when creating a class for reuse is that if an `equals()` method is specified for a class, then a consistent `hashCode()` method should also be specified. We will briefly review the rationale behind this, explaining what this consistency demands.

You do not need to know any details about hashcodes.

The method `hashCode()` is defined in `Object` as returning an `int` value, referred to as a **hashcode**. Such a value is used by Java in a technique called **hashing**, which speeds up searching a collection. For example, when storing elements in a collection such as a set, Java uses a series of storage compartments called **buckets** (see Figure 6).

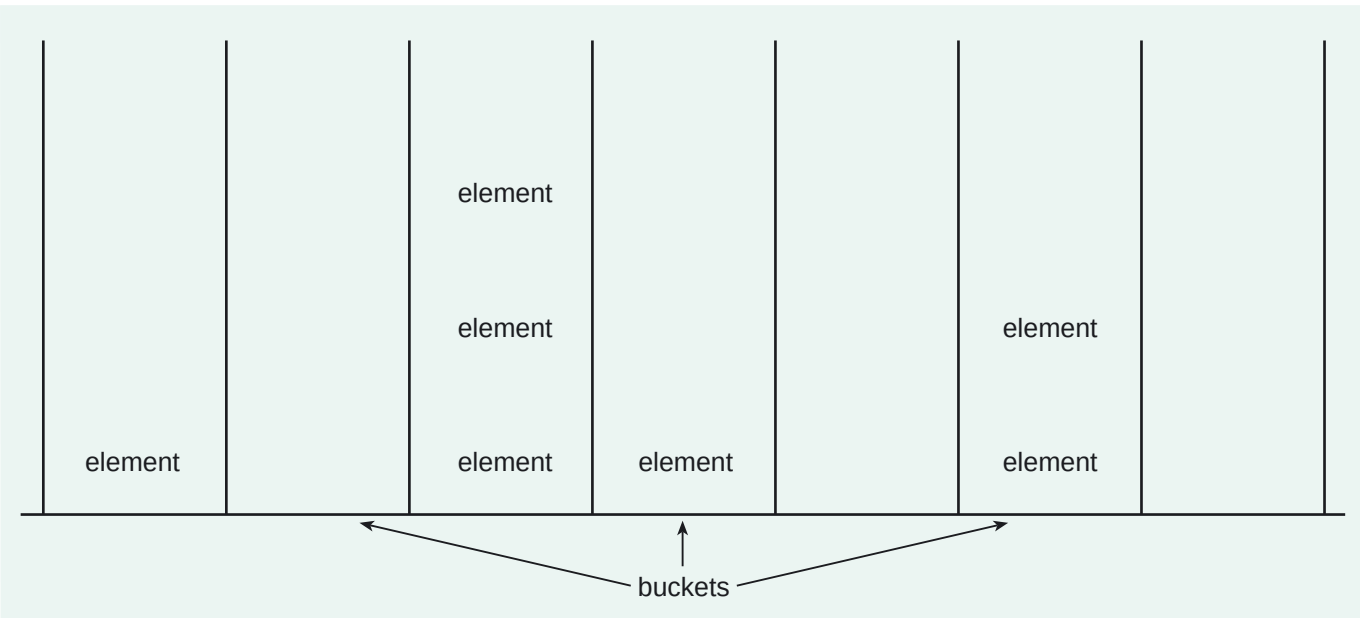


Figure 6 Storing elements in a set using buckets

When Java stores a new element in a set, the element's hashcode is used to determine the bucket in which it will be stored. The contents of that bucket are first checked to ensure that the new object will not duplicate any object already in there and, provided this is the case, the object is added to the bucket. In searching the set to determine whether an object is present, that object's hashcode is obtained and the corresponding bucket contents checked for the presence of the object.

This arrangement speeds up checking for the presence or absence of an object in a set, assuming buckets do not hold too many elements. However, there is a problem if `a` and `b`, say, are such that `a.equals(b)` returns `true` but `a.hashCode()` is not the same as `b.hashCode()`. Suppose that `a` is already in a set, and an attempt is made to add `b` to the set. Since `a.equals(b)` returns `true`, `a` and `b` are considered to be equal and therefore, since sets do not contain duplicate objects, `b` should not be added into the set. However, because `b`'s hashcode is different from that of `a`, `a`'s bucket is not searched; a different bucket, corresponding to `b`'s hashcode, is checked and `b` is placed into the set (see Figure 7).

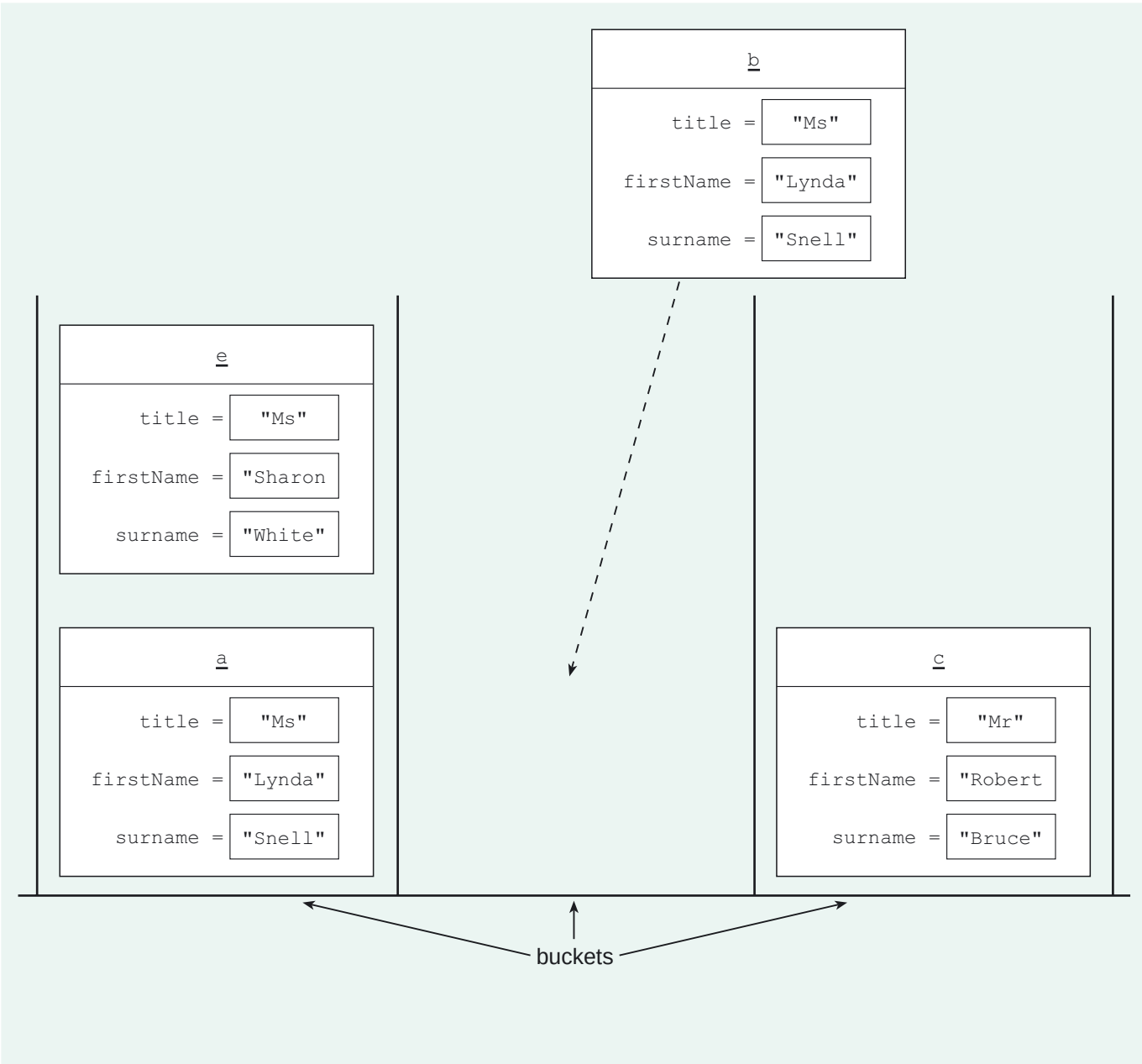


Figure 7 Storing duplicate `Name` objects in a set

To resolve this problem requires the designer to specify a `hashCode()` method for the class which is consistent with `equals()`, so that whenever `a.equals(b)` returns `true`, `a.hashCode()` and `b.hashCode()` are the same. That way, Java will use the same bucket for each object. The details of determining the hashcode can be left to the programmer; all that is required here is to adhere to this aspect of good practice and to specify the following method for the `Name` class:

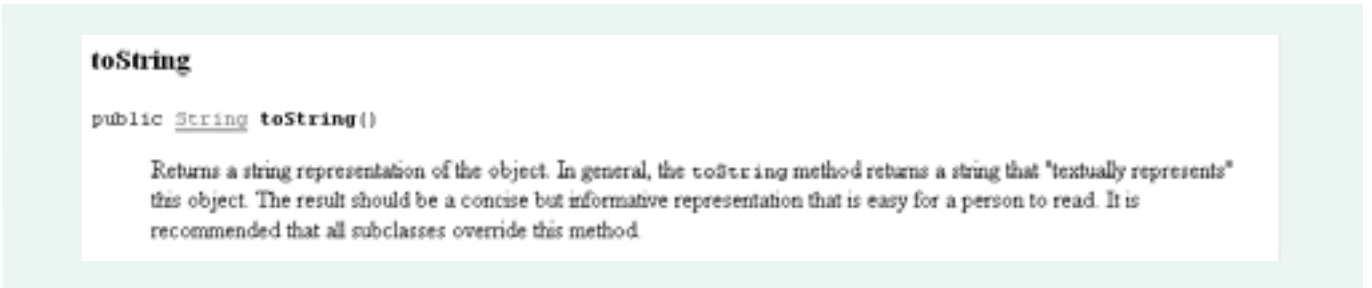
The converse need not hold. That is, it may be the case that `a.hashCode()` and `b.hashCode()` are the same, but `a.equals(b)` returns `false`.

```
public int hashCode()
```

Post-condition: returns the hashcode of the receiver. Consistent with `equals()`.

3.5 toString() for Name

Here is an excerpt from the documentation of the `toString()` method as it is specified for the class `Object`.



toString

`public String toString()`

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

Figure 8 Excerpt from the specification of `toString()` in `Object`

You can see from this specification that another aspect of good practice in design is to give each class a `toString()` method, which returns a meaningful textual representation of the receiver object. This is because all sorts of common programming tasks require an object to have an informative `toString()` method. For example, an object's `toString()` method is automatically invoked when the object is an argument to the `println()` method. It is usually sensible for a `toString()` method to return information about any attribute values of potential interest to a programmer.

Exercise 5

Specify a `toString()` method for the `Name` class.

Discussion.....

`public String toString()`

Post-condition: returns a string representation of the receiver's `title`, `firstName` and `surname`.

The specification of the `Name` class can now be completed.

Class `Name` — A person's name
This class is immutable. It implements `Comparable<Name>`

Attributes

`private String title`	The title of the person
`private String firstName`	The first name of the person
`private String surname`	The surname of the person

Constructor

`public Name(String aTitle, String aFirstName, String aSurname)`

Post-condition: initialises a new `Name` object with the given attribute values.

Protocol

`public String getTitle()`

Post-condition: returns `title`.

`public String getFirstName()`

Post-condition: returns `firstName`.

`public String getSurname()`

Post-condition: returns `surname`.

`public boolean equals(Object o)`

Post-condition: returns `true` if `o` is a `Name` object with `title`, `firstName` and `surname` equal to those of the receiver, otherwise returns `false`.

`public int hashCode()`

Post-condition: returns the hashcode of the receiver. Consistent with `equals()`.

`public int compareTo(Name aName)`

Post-condition: returns a negative integer if the receiver is alphabetically before `aName`, a positive integer if the receiver is alphabetically after `aName` and zero otherwise. `Name` objects are compared using `surname`, `firstName` and `title` in that order.

`public String toString()`

Post-condition: returns a string representation of the receiver's `title`, `firstName` and `surname`.

SAQ 9

In designing a class whose objects represent real-world entities that have a natural ordering and a notion of equality different from identity, what methods does good practice suggest should be specified for the class?

ANSWER.....

The following methods should be specified:

`compareTo()`, `equals()`, `hashCode()` and `toString()`.

3.6 Applying good practice to core system classes

The classes in our core system, such as `Ward` and `Team`, are less likely than a class such as `Name` to be reused individually. There would be little point in having a `Ward` class without the associated `Patient` class, for example: wards without patients would be useless. Instead, they are likely to be reused together, as part of the core system component.

However, it should still be considered whether any of the good practice mentioned in the context of the `Name` class should apply to the core system classes – `Ward`, `Team` etc. – in order to make reuse of the core system component more viable.

First, consider the possibility of specifying an `equals()` method to override the one inherited from `Object`. It is easy to see that for wards, for example, there is no sensible notion of equality other than identity: there is no natural concept of 'sameness' which would lead to two distinct `Ward` objects being considered to be the same. That is, the

`equals()` method inherited from `Object` is appropriate for `Ward`. In fact, there is no natural meaning of equality, other than object identity, for any of the core system classes. For example, there are no circumstances in which non-identical `Patient` objects should be considered to be the same. So each core system class will simply inherit `equals()` from `Object`.

Next, there is the issue of ordering to consider. We will take the class `Patient` as a typical example. People are sometimes sorted by alphabetical order of their name. But a `compareTo()` method for `Patient`, for example, specified so that `Patient` objects were compared by comparing their `name` attribute values, would be inconsistent with the `equals()` method which `Patient` inherits from `Object`.

SAQ 10

Suppose that a `compareTo()` method were specified for `Patient`, as follows.

```
public int compareTo(Patient aPatient)
```

> Post-condition: returns a negative integer if the receiver's `name` is alphabetically before `aPatient`'s `name`, a positive integer if the receiver's `name` is alphabetically after `aPatient`'s `name`, and zero otherwise.

Explain how this `compareTo()` method is inconsistent with `equals()` for `Patient`, which is such that for `p` and `q` referencing `Patient` objects, `p.equals(q)` returns `true` if and only if `p` and `q` refer to the same object.

ANSWER

Suppose `p` and `q` reference distinct `Patient` objects which happen to have the same value for `name` (that is, the objects represent distinct patients with the same name). Then `p.comparesTo(q)` returns `0`, but `p.equals(q)` returns `false`.

In fact, there is no single appropriate ordering for patients. Another client might want to order patients by name and date of birth (that is, where two patients have the same name, their dates of birth are compared). Such an ordering would lead to the same problem described in SAQ 10. For such reasons it is inappropriate to impose an ordering on `Patient`. Similar arguments apply to all the other core system classes: we will not specify a `compareTo()` method for any of them.

Exercise 6

We have decided that each of the core system classes will inherit `equals()` from `Object`, and that none of them will have `compareTo()` methods specified. But which of the other 'good practice' methods described above should be specified for the core system classes?

Discussion

Since `equals()` is not overridden, there is no need to specify an overridden `hashCode()` method either.

You can confirm that the Hospital System implementation model (Section 5 of the *Handbook*) specifies a `toString()` method for each class.

However, it is appropriate to specify an overridden `toString()` method for each class, since it is reasonable for a client to want a textual representation of a core object.

We have decided not to specify a `compareTo()` method for any core system class. However, the lack of a `compareTo()` method for `Patient`, for example, does not *prevent* a client from ordering `Patient` objects in some way appropriate to their particular needs: by name, or by name and date of birth, or in some other way. There are means by

which a client can arrange Patient objects in different orders, without using those aspects of collections which require elements to have a compareTo() method. We will not pursue the details of this here.

In this section you learnt about the representation of names and the decision to introduce the utility class Name. Careful consideration was given to which methods Name should be equipped with (given that it is a prime candidate for reuse by future projects), and we applied general principles of good practice relevant to specifying for reuse.

In the next section you will see how, in the process of reviewing the Hospital System designs, you can specify another class that has significant reuse potential and that also enables the structure of the system to be improved.

4 Refactoring

At this point in the development of the Hospital System, the core system classes and an appropriate utility class (`Name`) have, with reuse in mind, been designed and specified, as have two enumerated types for representing particular attributes. As a developer, you would now be in a reasonable position to start implementing and testing the code (or possibly, in a real project, handing over the designs to the programmers for implementation). However, a key detailed design task remains: to review the designs for the core system's structure as a whole with a view to **refactoring**, that is, amending the system's structure, while retaining its behaviour, in order to improve its quality – specifically, its reusability and maintainability.

In this section, you will see:

- how the designs for a system might be improved by refactoring;
- how refactoring can lead to the specification of additional utility class.

4.1 The `Person` class

We start by reviewing the structural part of the implementation model as it has been developed so far. Here is part of the model which gives excerpts from the class descriptions for the classes `Patient`, `Doctor`, `JuniorDoctor` and `ConsultantDoctor` (including only the specification of attributes, and some methods).

Class `Patient` — A patient in the hospital

Attributes

`private Name name`	The name of the patient
`private Sex /sex`	The sex (M or F) of the patient
`private M256Date dateOfBirth`	The date of birth of the patient
`[int /age`	The age of the patient in years]

Protocol

`public Name getName()`
Post-condition: returns `name`.

`public Sex getSex()`
Post-condition: returns `sex`.

`public int getAge()`
Post-condition: returns `age`.

`public M256Date getDateOfBirth()`
Post-condition: returns `dateOfBirth`.

...

Class `Doctor` — A doctor at the hospital
Abstract: generalises `JuniorDoctor` and `ConsultantDoctor`

Attributes

`private Name name` — The name of the doctor

Protocol

`public Name getName()`

Post-condition: returns `name`.

...

Class `JuniorDoctor` — A junior doctor at the hospital
Specialises `Doctor`

Attributes

`private Grade grade` — The grade (1, 2 or 3) of the junior doctor

Protocol

`public Grade getGrade()`

Post-condition: returns `grade`.

...

Class `ConsultantDoctor` — A consultant doctor at the hospital
Specialises `Doctor`

Attributes

None

Protocol

None

Exercise 7

In the above extract, identify any features that are common to two or more classes.

Discussion.....

Both `Patient` and `Doctor` have an attribute, `name`, and a method, `getName()`, whose specifications are identical. `JuniorDoctor` and `ConsultantDoctor` have these features too, by virtue of specialising `Doctor`.

`Patient` and `Doctor` actually have more than just these structural elements – an attribute and a method – in common. They each represent entities (patients and doctors) which are conceptually similar; they are both roles that are played by *people*. Now, this similarity was not of particular significance to the real-world system domain of the hospital: people, per se, were not explicitly described in the requirements document, and doctors and patients were considered to be entirely separate entities when creating a conceptual model of the system domain. But during design, such conceptual similarities, which may be indicated as here by classes having attributes and methods in common, can be used as a basis for improving the design.

In this case, we will specify an additional class, called `Person`, to represent the common features of doctors and patients in the hospital. To begin with we will assign `Person` an attribute, `name`, and a getter method, `getName()`, as follows.

Class `Person` A person

Attributes

`private Name name` The name of the person

Protocol

`public Name getName()`

Post-condition: returns `name`.

Normally the features 'factored out' in this way would be more significant, but here we are using an artificially simple example to get the essential ideas across.

Using a class to explicitly represent the common features of people who are doctors and patients improves the system because it more accurately represents the real world. That makes it more understandable and therefore more easily maintainable. Depending on how the Hospital System uses the `Person` class, there are other particular benefits, which will be discussed later in this section.

Note that having attributes and methods in common is not sufficient grounds for specifying an additional class with these common elements. `Patient` objects and `Doctor` objects have `name` attributes and `getName()` methods which are similar, because both classes model conceptually similar entities. On the other hand, `Ward` objects, which also have a `name` attribute and a `getName()` method (with different specified types), do not model entities which are conceptually similar to either patients or doctors: wards are quite a different kind of entity.

As with names, discussed in Section 3, the need to represent people is common throughout computing, and it is likely that, if `Person` is designed carefully, it could be reused in other systems. That is, it is a utility class. The utility class `Name` was equipped with methods likely to be of use to future developers. These extra methods were not required by the Hospital System, but can be ignored by it since it employs `Name` objects in relatively simple ways. With the class `Person`, the ways in which it will be used by the Hospital System have yet to be considered. Only then can we decide whether it is appropriate to specify additional functionality for this class.

So, how might the Hospital System use the `Person` class? That is, how could `Doctor` and `Patient` be respecified to take advantage of the fact that the common features of the entities they represent can now be represented by `Person` objects?

Two options will be evaluated: *inheritance* and *composition*.

4.2 Inheritance

One way in which `Person` could be used is to specify `Doctor` and `Patient` as subclasses of `Person`. The attribute `name` and the method `getName()` would be inherited from the `Person` superclass and would no longer be specified in `Doctor` or `Patient`.

When introducing a superclass during design it is usually important to confirm that the relationship between the classes is consistent with the conceptual relationship between the real-world entities. Otherwise the system becomes a poor representation of the real world, making it less understandable and harder to maintain. In this case, since doctors and patients can both be thought of as kinds of people, introducing `Person` as a superclass of `Doctor` and `Patient` is a reasonable thing to do.

SAQ 11

Why would it not be appropriate to introduce `Person` as a superclass of `Ward`?

ANSWER

This is not appropriate because a ward is not a kind of person.

You might also have noted that `Ward`'s `name` attribute is specified to be of a different type (`String`) from that of the `name` attribute of `Person` (`Name`), and that there are corresponding differences in the return type of the 'common' method `getName()`. These features are symptomatic of the fundamental difference between wards and people.

4.3 Composition

If *composition* is used, the relationship between the classes is quite different. Composition is a technique based on including one object (`q` in Figure 9) within the state of another (`p`). In Java programming, this technique is commonly used without a second thought by developers, yet it can be immensely powerful.

You were introduced in *Unit 5*, Subsection 7.1, to the idea of reusing a class by composition.

Figure 9 `p` uses `q` by composition

SAQ 12

Give two examples of composition from the Hospital System designs so far. (Look at the class descriptions at the beginning of Subsection 4.1.)

ANSWER

Composition is at work whenever a class has an instance variable that references an object. For example:

- `Patient` has an attribute, `name`, which is implemented by an instance variable of type `Name`. `Patient` objects thus use `Name` objects by composition.
- `Patient` also has an attribute, `dateOfBirth`, which is implemented by an instance variable of type `M256Date`. `Patient` objects thus use `M256Date` objects by composition.

The real force of composition becomes apparent in situations more complex than those mentioned in the answer to SAQ 12, specifically when a first object, `p`, which uses a second object, `q`, by composition, carries out some of its work by forwarding it to `q`. A **forwarding method** of `p` is a method which simply sends (forwards) the corresponding message to `q`. Often (but not always) the methods involved have the same name, as illustrated in Figure 10.

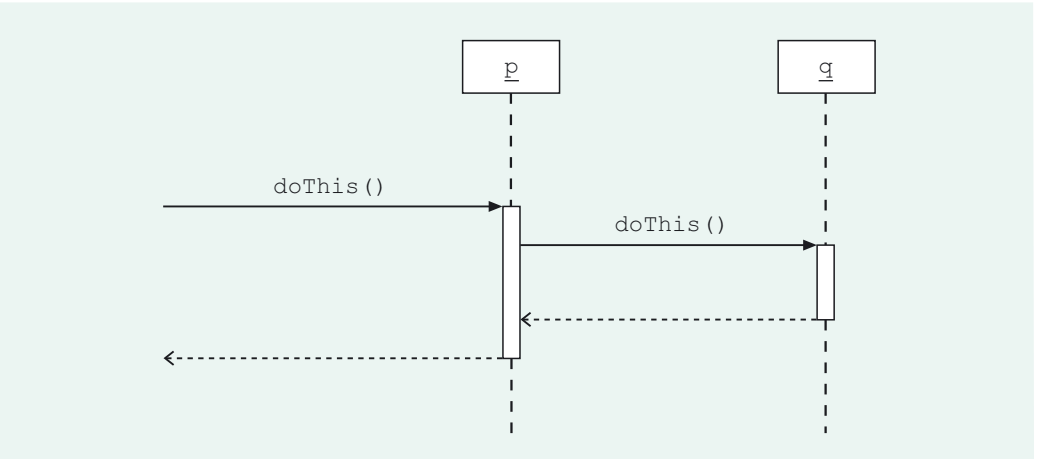


Figure 10 `p` forwards a message to `q`

You will now see how `Doctor` and `Patient` could each use `Person` by composition.

In this case, instead of each of `Doctor` and `Patient` inheriting from the `Person` class, they would each specify an attribute, `person`, implemented by an instance variable of type `Person`. The specification of the `name` attribute in each of `Doctor` and `Patient` would be removed, and the `getName()` method in each class would be a forwarding method, with the `Person` object referenced by `person` being called on to provide the name to be returned by the method.

To help you understand this idea, Exercise 8 will ask you to look ahead to the implementation phase, and consider some potential code. First, here is the specification for the `Doctor` class corresponding to the composition arrangement just described.

Class `Doctor` — A doctor at the hospital
Abstract: generalises `JuniorDoctor` and `ConsultantDoctor`

Attributes

`private Person person` — The person who is a doctor

Protocol

`public Name getName()`

Post-condition: returns `name`.

Exercise 8

Imagine that development has moved on a phase, and that you, as a programmer, are implementing the core system on the basis of class descriptions such as those above. On the basis of the `Person` and `Doctor` class descriptions, write code for the body of the `Doctor` method `getName()`.

Discussion

The code should return the value of the receiver's `Person` object's `name`, as follows.

```
return person.getName();
```

Composition corresponds to taking a different view of the relationship between the real-world entities involved from that in inheritance. Having `Doctor` use `Person` by composition corresponds to thinking of a doctor not as a *variety* of person, but as intrinsically a person who is also taking on the role of a doctor (but who in other circumstances might take on other roles). Informally, the functionality specific to a

Doctor object is 'wrapped around' its inner Person object. If an instance of Doctor is sent a message that is understood by its inner Person object, it forwards it to the latter. The message answer is passed back to the Doctor object, which in turn relays it to the sender of the original message. In this way Doctor draws on Person to manage all the person-like behaviour involved in being a doctor.

Exercise 9

Think back to the conceptual modelling phase, in *Unit 3*, when the real-world context of the Hospital System was modelled. It was decided that it was appropriate for each of the conceptual classes JuniorDoctor and ConsultantDoctor to specialise the conceptual Doctor class, because both the represented categories of entity – junior doctors and consultant doctors – are kinds of doctor.

However, this does not mean that the designer is obliged to specify the software classes JuniorDoctor and ConsultantDoctor as subclasses of Doctor. Although JuniorDoctor and ConsultantDoctor *will* be implemented as subclasses of Doctor (in *Unit 10*), it is instructive to consider an alternative. The questions below guide you through this.

(a) How else other than by inheritance could JuniorDoctor and ConsultantDoctor use the Doctor class?

(b) Describe the relationship between doctors to which this alternative use of the Doctor class corresponds.

(c) Revise the excerpt from the JuniorDoctor class description, given at the beginning of Subsection 4.1, according to this alternative use of the Doctor class.

(d) What change would be required to the Doctor class for this alternative use?

Discussion

(a) They could each use Doctor by composition.

(b) This corresponds to thinking of a junior doctor as a doctor who is taking on the role of a junior, and of a consultant doctor as a doctor who is taking on the role of a consultant.

(c) The JuniorDoctor class description is as follows.

Class JuniorDoctor — A junior doctor at the hospital

Attributes

`private Grade grade`	The grade (1, 2 or 3) of the junior doctor
`private Doctor doctor`	The doctor who is a junior

Protocol

`public Grade getGrade()`

Post-condition: returns `grade`.

`public Name getName()`

Post-condition: returns `name`.

(d) Doctor could no longer be an abstract class, since instances of JuniorDoctor and ConsultantDoctor would have to reference instances of the class Doctor.

4.4 Composition and inheritance compared

Exercise 8 showed that composition can be implemented very straightforwardly, but we have noted that it can also be very powerful. Inheritance is likewise a powerful and often-used technique, so it is important to understand some of the relative advantages and disadvantages of inheritance and composition.

We will, in fact, choose to use `Person` by composition in the Hospital System, and we will now proceed to discuss this choice and explain some of the factors involved in evaluating choices in this area. This is a complex (and much debated) area, so the discussion will be restricted to some key points in the context of the Hospital System.

Reuse potential

Using composition offers a distinct advantage for the reuse potential of the class `Person`. Imagine that, as for `Name`, you want to equip `Person` with certain basic methods to make it a more generally useful class. You might think of specifying that `Person` objects should manage information related to a person's address, and specify methods such as `getAddress()`. Having `Doctor` and `Patient` inherit from `Person` renders such extra functionality inappropriate because `Doctor` and `Patient` would inherit it, and there is no requirement in the Hospital System for addresses of doctors and patients to be handled.

Using `Person` by inheritance would mean that it would be inappropriate for `Person` objects to manage even such basic information as a person's sex, date of birth and age. Although it would be appropriate for `Patient` to inherit such behaviour from `Person`, there is no requirement for the Hospital System to deal with such behaviour in relation to doctors.

On the other hand, if composition is used, then all sorts of potentially useful functionality can be specified for `Person` without impacting on the system under development. `Doctor` and `Patient` can make use of those parts of the `Person` class that are appropriate for them, by specifying forwarding methods, and simply ignore the rest.

Since we are choosing to use `Person` by composition, we will specify `Person` with `sex`, `dateOfBirth` and `age` attributes, as well as `name`. `Doctor` will call on `Person`'s name-related behaviour, and ignore the rest; `Patient` will use all of these aspects of `Person`.

Future flexibility

If `Doctor` and `Patient` were subclasses of `Person`, then the system could not easily be extended in the future to represent a doctor in the hospital being admitted as a patient, because the same object cannot represent both a doctor and a patient. If a person were both a patient and a doctor, this person would be represented by two distinct objects, one representing the person in the role of the doctor, and the other representing the person in the role of the patient. The 'personal' data, such as name, would be recorded twice, and there would be no way of recording that the two objects represented roles played by the same person.

On the other hand, if composition is used and a person is both a doctor and a patient, then the situation is much simpler. When a doctor, represented by a `Doctor` object (i.e. a `JuniorDoctor` object or a `ConsultantDoctor` object) is admitted as a patient, a new `Patient` object can be created which references the same `Person` object as does the `Doctor` object. The person involved is represented by an instance of `Person` which is common to both a `Doctor` and a `Patient` object, as illustrated in Figure 11.

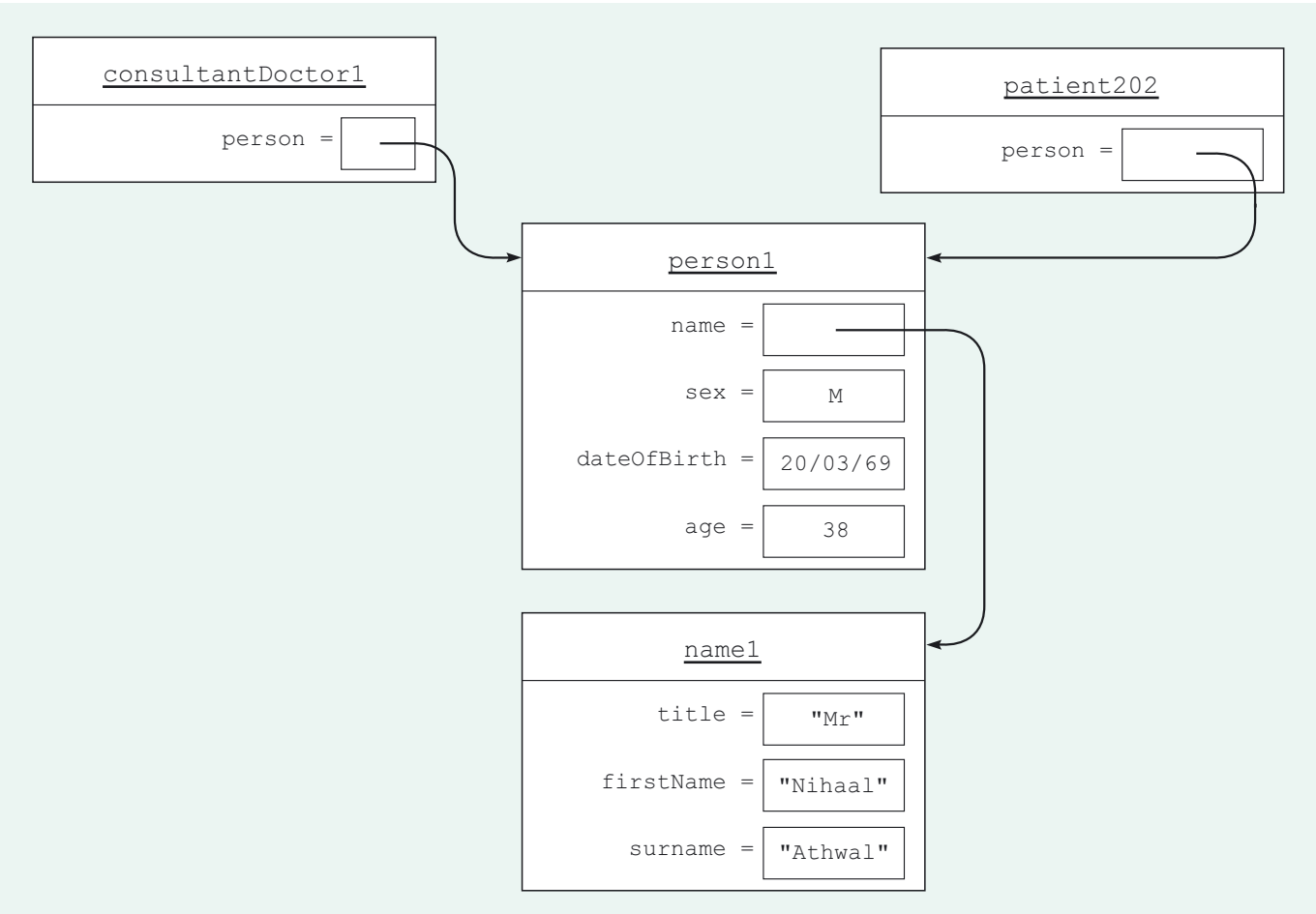


Figure 11 A `Doctor` object and a `Patient` object share a `Person` object

The `age` attribute reflects the situation in 2007.

Reducing code duplication

Both inheritance and composition contribute to reducing code duplication, giving the following particular benefits:

- more reuse within the system, since a single class is used to represent people, rather than these people-related features being designed separately into both `Patient` and `Doctor`;
- simplification of the structure of the system, making it more understandable and easily maintainable;
- reduced programmer workload.

Exercise 10

Describe how each of the following situations would reduce code duplication between the classes `Doctor` and `Patient`, as they were originally specified.

(a) Having `Doctor` and `Patient` inherit from `Person`.

(b) Having `Doctor` and `Patient` use `Person` by composition.

Discussion.....

Both `Doctor` and `Patient` originally specify an attribute, `name`, and a method, `getName()`.

(a) If `Doctor` and `Patient` inherited from `Person`, then neither of these classes would implement a `name` instance variable or a `getName()` method. These elements would instead be implemented in the `Person` class.

(b) With `Doctor` and `Patient` using `Person` by composition, neither class needs to implement a `name` instance variable, this being implemented instead in `Person`. However, there would be no saving on code for methods, since both would still need `getName()` methods (although in this case they would be forwarding methods).

Since composition requires the use of forwarding methods, code that makes use of composition inevitably contains more duplication than that using inheritance. That is, the technique of composition reduces code duplication, but not as much does inheritance.

Using the `Person` class by composition

Having compared some of the issues in the inheritance versus composition debate, you are now in a position to see that the decision to use `Person` by composition allows the specification of a `Person` class so that:

- it can be used appropriately in restructuring the Hospital System;
- it is attractive to future development projects;
- any of its features that are superfluous to the Hospital System can simply be ignored.

Exercise 11

What services might you expect a `Person` class to offer, in addition to managing information about a person's name, sex, date of birth and age?

Discussion.....

There is no one correct answer to this question.

Here are some additional services you might have thought of:

- managing information about a person's address, telephone number, email etc.;
- a means of creating `Person` objects initialised in different ways;
- a means of obtaining a textual representation of a `Person` object.

You might also have included equality and ordering as services. The reason we have not done so here is discussed below.

As shown in Exercise 11, there are many interesting and potentially useful services that could be specified for `Person`. For simplicity, the following will be taken as the specification of `Person`.

Class	`Person`	A person
	Attributes	
	`private Name name`	The name of the person
	`private Sex sex`	The sex (M or F) of the person
	`private M256Date dateOfBirth`	The date of birth of the person
	`[int /age`	The age of the person in years]

Constructors

`public Person(Name aName, Sex aSex, M256Date aDate)`

Post-condition: initialises a new `Person` object with the given attribute values and `age` according to `aDate`.

`public Person(Name aName)`

Post-condition: initialises a new `Person` object with the given `name` value. Sets `sex`, `dateOfBirth` and `age` to `null`.

`public Person(Person aPerson)`

Post-condition: initialises a new `Person` object with the same attribute values as `aPerson`.

Protocol

`public Name getName()`

Post-condition: returns `name`.

`public Sex getSex()`

Post-condition: returns `sex`.

`public int getAge()`

Post-condition: returns `age`.

`public M256Date getDateOfBirth()`

Post-condition: returns `dateOfBirth`.

`public Name setName(Name aName)`

Post-condition: sets `name` to `aName`.

`public String toString()`

Post-condition: returns a string representation of the receiver's `name`, `sex` and `dateOfBirth`.

There are a number of features to note about this specification.

The attribute `age` is derived

Just as was the case for the `Patient` class, the attribute `age` of `Person` is marked as derived (`/`), because the value of `age` can be calculated from the value of `dateOfBirth`. This derivation means that `age` need not be implemented as an instance variable, a decision denoted by [...].

Unlike the original `Patient` specification, `sex` is not derived. For `Patient`, `sex` was derived using the following invariant:

> The `sex` attribute of any `Patient` object has the same value as the `type` attribute of the linked `Ward` object.

However, `sex` in `Person` cannot be derived in this way, since `Person` objects are not linked to `Ward` objects.

`Person` has no `equals()` or `compareTo()` method

Note first that it is appropriate for `Person` *not* to override the `equals()` method inherited from `Object`. No two distinct `Person` objects should be considered equal, just as we discussed earlier for `Patient`. So no `equals()` method is specified for `Person`.

In addition, no ordering has been specified for `Person` objects: there is no `compareTo()` method. Various orderings might be considered: by alphabetical order of name, by age etc. Since there is no one obvious ordering, it is not sensible to impose one on the class.

However, there is another, related reason why specifying an ordering is a poor idea. This is explored in SAQ 13.

SAQ 13

If `a` and `b` reference `Person` objects, then `a.equals(b)` returns true only when `a` and `b` reference the same object. On this basis, explain why we should not specify an ordering on `Person` based on `name`.

ANSWER.....

If a `compareTo()` method is defined, it should be consistent with `equals()`. In particular, `a.compareTo(b)` should return `0` only when `a.equals(b)` returns `true`, that is, only when `a` and `b` reference the same object.

But it is entirely possible that distinct objects may have the same value for `name`, so that `a.compareTo(b)` returns `0` while `a.equals(b)` returns `false`.

`Person` is not immutable

People's personal details do change, so it is not appropriate for `Person` to be an immutable class. However, for simplicity, just one state-changing method, the setter `setName()`, has been specified, rendering `Person` objects *mutable*.

`Person` constructors

In *Unit 5*, Subsection 6.1, you learnt how a copy constructor can be used to protect mutable objects. Since `Person` objects are mutable, there is a possibility that a client that has a reference to a `Person` object can make changes to it, perhaps as the result of a programming error, thereby potentially putting it into an incorrect state. A copy constructor takes the original object as an argument and creates a copy, which clients can then use without being able to modify the original. For this reason a copy constructor is included in the above specification of the `Person` class.

SAQ 14

Which is the copy constructor for `Person`?

ANSWER.....

The constructor with header `public Person(Person aPerson)` is the copy constructor, since it takes a `Person` argument and initialises a new `Person` object with the same state as `aPerson`.

Two other constructors have been defined for `Person`.

The first, with header `public Person(Name aName, Sex aSex, M256Date aDate)`, initialises a new `Person` object with supplied values for `name`, `sex` and `dateOfBirth`. This will be used in creating new `Patient` objects.

The second, with header `public Person(Name aName)`, initialises a new `Person` object with a supplied value for `name`, and sets `sex` and `dateOfBirth` to `null`. This will be used in creating new `Doctor` objects, since neither the sex nor the date of birth of a doctor should be represented.

Impact on `Patient`

Specifying the `Person` class like this means that the `Patient` class no longer has to specify the attributes `name`, `sex`, `dateOfBirth` and `age`. Each `Patient` object's `Person` object is instead responsible for maintaining this information.

SAQ 15

Rewrite the **Attributes** section of the `Patient` class description.

ANSWER

Attributes

`private Person person`	The person who is a patient

4.5 `hospitalcore` and `m256people`

The enumerated types `Grade` and `Sex`, and the utility classes `Name` and `Person`, have been specified as part of finalising the design for the Hospital System. Conceptually, these classes are quite different from those in the core system, such as `Ward` and `Team`. The latter represent significant categories of entity in the real-world system domain, whereas `Name`, `Person` and, more loosely, `Grade` and `Sex` are quite general, representing concepts – people for example – which are not specific to the system domain and instead may be applicable more widely.

For this reason it is not appropriate to include them in the core system, which will be implemented in a package called `hospitalcore`. Instead, they will be housed together in a component, which we will implement as a package called `m256people`.

Exercise 12

Review the Hospital System implementation model, in Section 5 of the *Handbook*. Compare the structural model that was the starting point for this unit (see the Appendix to *Unit 7*) with the completed structural model (specifically, the class descriptions) shown in Section 5 of the *Handbook*, and list all the changes that have been made, confirming that they are consistent with those discussed in this unit.

Discussion

1 The completed model has a `hospitalcore` package in which the core system classes are specified.

2 `Patient` now has an attribute `person` of type `Person` in place of the attributes `name`, `sex`, `dateOfBirth` and `age`.

3 `Doctor` has an attribute of type `Person` in place of the attribute `name`.

4 Each core system class specifies a `toString()` method.

5 There is a new package called `m256people` that contains the new utility classes `Name` and `Person`, and the enumerated types `Grade` and `Sex`.

In this section, a new utility class, `Person`, has been specified, and its use by inheritance and composition contrasted. This has led to changes to the `Patient` and `Doctor` specifications in the implementation model, which now also specifies the package `m256people`, in which `Person` is housed, as well as `hospitalcore`, the package in which the core system will be implemented. This implementation model is the basis on which coding and testing will be carried out, in *Unit 10*.

5 Summary

In this unit you have been introduced to the sorts of decision that need to be taken before the implementation model can be used as a basis for coding and testing. These decisions included:

- deciding which existing types to specify for instance variables that implement attributes and link references, and for method answers;
- specifying new user-defined types (two enumerated types and two classes were specified for the Hospital System);
- promoting the reuse of new classes by considering not only their role in the current system but also possible future uses to which they might appropriately be put;
- refactoring the structure of the system to improve its quality.

Naturally, since Java is our target language, the ideas we have discussed in this unit have been grounded in Java. However, everything we have discussed has parallels in other object-oriented languages, though the details are different.

LEARNING OUTCOMES

After studying this unit you should be able to:

- describe and use each of this unit's key terms (summarised in the Glossary);
- decide which existing types to specify for instance variables which implement attributes and link references, and for method answers;
- specify new user-defined types for use in the current system, which have the potential for reuse by other systems and apply established good practice appropriately;
- refactor the structure of a system, using inheritance and composition, to improve its quality.

Glossary

buckets Storage compartments used when storing elements in a collection, such as a set.

enumerated type (enum) A user-defined Java type that can take on only one of a finite set of values specified by the programmer when the type is defined.

forwarding method A method in which the corresponding message is forwarded to an object being used by composition.

hashcode A value returned by a `hashCode()` method, used by Java in hashing.

hashing A technique that speeds up searching a collection.

interface type An interface declared as the type of a variable or method answer.

refactoring The process of amending a system's structure, while retaining its behaviour, in order to improve its quality.

user-defined type A type defined by the user, augmenting those provided by the Java API.

utility class A class which is not specific to the particular system under development, but which has potentially wider applicability.

Index

B
bucket 20

C
class flexibility 32
code duplication 33
`Comparable` 18

D
dynamic part (of implementation model) 5

E
enumerated type (enum) 12

F
forwarding method 29

H
hashcode 20
`hashCode()` 20
hashing 20

I
immutable class 17
immutable object 17
implementation model 5, 37
interface type 9

J
Java types 12

M
maintainability 6

N
`Name` 15

P
`Person` 34

R
refactoring 26
reuse 6, 32

S
state 17
structural model 5
subclass 28
superclass 28

U
user-defined type 11
utility class 15